

Employee Data Warehouse

Multi-Source ETL Pipeline

Project Documentation

Project Type	Data Warehousing
Tools Used	SQL Server · SSIS
Schema	Star Schema
Author	Hadeer Badr

Table of Contents

Table of Contents	2
1. Project Overview.....	3
1.1 Business Problem.....	3
1.2 Objectives	3
2. Project Architecture.....	4
2.1 Layer Responsibilities.....	4
3. Tools & Technologies.....	5
4. Data Warehouse Design — Star Schema	6
4.1 Data Model Diagram.....	6
4.2 Fact Table	7
FactEmployeeSalary.....	7
4.3 Dimension Tables.....	8
DimEmployee	8
DimDep — Department Dimension	8
DimILoc — Location / Country Dimension	8
DIMGender — Gender Dimension	8
5. ETL Process (SSIS).....	9
5.1 Extraction	9
5.2 Transformation	10
5.3 Loading	12
6. SQL Development.....	13
6.1 Stored Procedure — Dim_Date Population	13
DimDate Output Columns	14
Sample Validation Queries.....	14
7. Challenges & Solutions	15
8. Future Enhancements.....	16
9. Conclusion	17

1. Project Overview

1.1 Business Problem

Organizations rarely store employee data in one place. HR systems export CSV files, payroll and legacy ERP platforms export XML, and internal APIs deliver employee records as JSON. Each of these systems uses its own naming conventions, its own date formats, and its own data quality standards — and none of them agree with each other.

This fragmentation creates a direct business problem: leadership cannot get a single, trustworthy answer to simple questions like “How many employees do we have in each country?” or “What is our average salary by department?” without an analyst manually reconciling three different spreadsheets, each with different column names, different formats, and different errors.

The Employee Data Warehouse project was built to solve exactly this problem: to take three structurally different, individually flawed data sources and deliver one clean, trusted, query-ready data warehouse that any analyst or BI tool can use with confidence.

1.2 Objectives

- Consolidate employee data from three heterogeneous source systems (CSV, JSON, XML) into a single warehouse
- Preserve full traceability of raw source data through a staging layer before any transformation is applied
- Identify and resolve all data quality issues — inconsistent formats, typos, type mismatches — before data reaches the warehouse
- Design and implement a Star Schema data model optimized for fast, simple analytical queries
- Build a repeatable, automated ETL pipeline using SQL Server Integration Services (SSIS)
- Validate incoming XML data against its XSD schema to catch structural issues at the point of ingestion
- Deliver a data warehouse that supports workforce analytics: headcount, salary distribution, and demographic reporting

2. Project Architecture

The project follows a layered data warehouse architecture, moving data through three distinct stages — each with a clear, single responsibility. This separation of concerns is what makes the pipeline maintainable, auditable, and safe to re-run.

ODS LAYER	STG LAYER	DWH LAYER
Raw operational mirror No transformation Source data preserved as-is	Staging.dtsx Per-run landing zone Source-tracked, audited	DWH.dtsx Star Schema Fact + 4 Dimensions

Sources (CSV / JSON / XML) → ODS → STG → DWH (Star Schema) → BI Tools

2.1 Layer Responsibilities

Layer	Purpose	Transformation Applied
ODS	Operational mirror of source systems for traceability and recovery	None
STG	Per-run landing zone with audit metadata (load timestamp, source file, row count)	None
DWH	Clean, business-ready Star Schema for analytics and reporting	Full cleansing & standardization

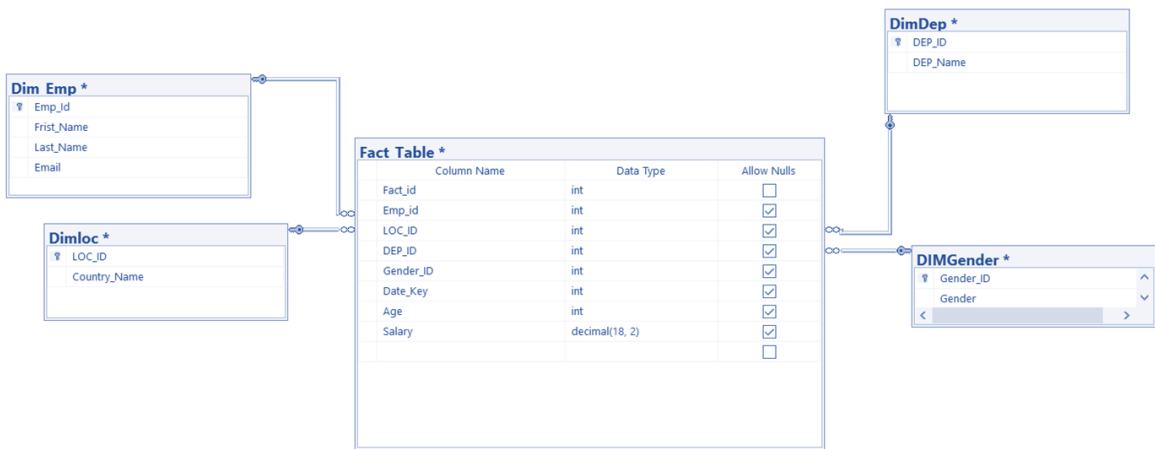
3. Tools & Technologies

The project was built entirely on the Microsoft Data Platform, using industry-standard tools for ETL development and data warehousing.

Tool	Category	Role in This Project
SQL Server Express	Database Engine	Hosts ODS, STG, and DWH databases (localhost\SQLEXPRESS)
SQL Server Integration Services (SSIS)	ETL Tool	Builds and executes the ODS, STG, and DWH packages
Visual Studio + SSDT	Development Environment	Authoring and debugging .dtsx SSIS packages
SQL Server Management Studio (SSMS)	Database Administration	Schema design, relationships, validation queries
XML Schema Definition (XSD)	Data Contract	Validates structure of incoming XML source files
T-SQL	Query Language	Stored procedures, transformations, validation queries
Git / GitHub	Version Control	Source control and project hosting

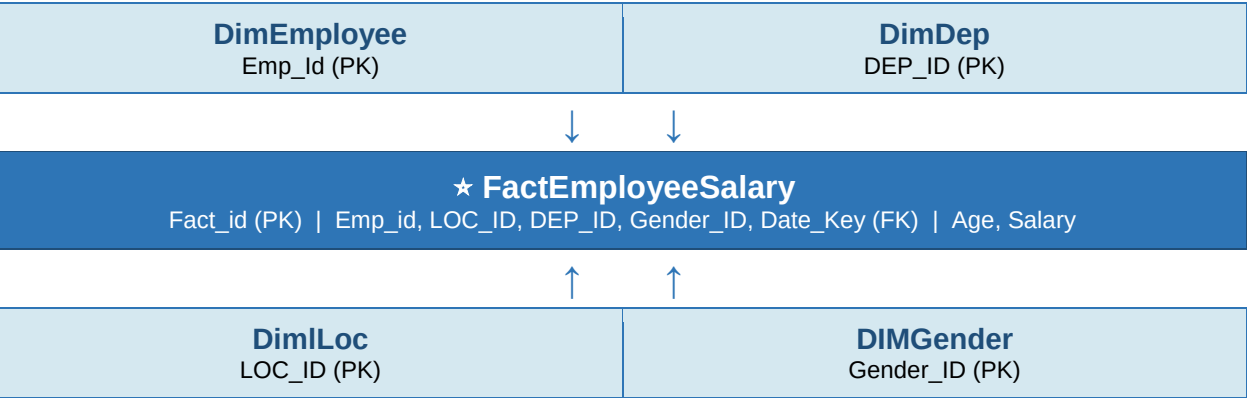
4. Data Warehouse Design — Star Schema

The Data Warehouse layer implements a Star Schema following the Kimball dimensional modeling methodology. A single fact table holds measurable business events (employee salary records), surrounded by four dimension tables that describe the who, what, and where of each record.



4.1 Data Model Diagram

The model consists of one central fact table (FactEmployeeSalary) connected to four dimension tables via foreign key relationships:



Note: A DimDate table is recommended to fully resolve Date_Key in the fact table (see Future Enhancements, Section 8).

4.2 Fact Table

FactEmployeeSalary

Column	Data Type	Allow Nulls	Description
Fact_id	INT	NO	Primary key (surrogate, identity)
Emp_id	INT	YES	Foreign key → DimEmployee
LOC_ID	INT	YES	Foreign key → DimLoc
DEP_ID	INT	YES	Foreign key → DimDep
Gender_ID	INT	YES	Foreign key → DIMGender
Date_Key	INT	YES	Foreign key → DimDate (YYYYMMDD format)
Age	INT	YES	Employee age at time of record
Salary	DECIMAL(18,2)	YES	Monthly salary, cleaned from source (\$ stripped, cast to decimal)

4.3 Dimension Tables

DimEmployee

Column	Data Type	Notes
Emp_Id	INT PK	Surrogate key (identity)
First_Name	VARCHAR(100)	Standardized from CSV first_name, JSON full_name (split), and XML first_name
Last_Name	VARCHAR(100)	Standardized across all 3 sources
Email	VARCHAR(255)	Source-system identifier, used for deduplication

DimDep — Department Dimension

Column	Data Type	Notes
DEP_ID	INT PK	Surrogate key
DEP_Name	VARCHAR(100)	Department name, e.g. Engineering, Sales, Human Resources

DimLoc — Location / Country Dimension

Column	Data Type	Notes
LOC_ID	INT PK	Surrogate key
Country_Name	VARCHAR(100)	Standardized and deduplicated country names from all 3 sources

DIMGender — Gender Dimension

Column	Data Type	Notes
Gender_ID	INT PK	Surrogate key
Gender	VARCHAR(10)	Standardized to Male / Female across all sources (CSV used M/F)

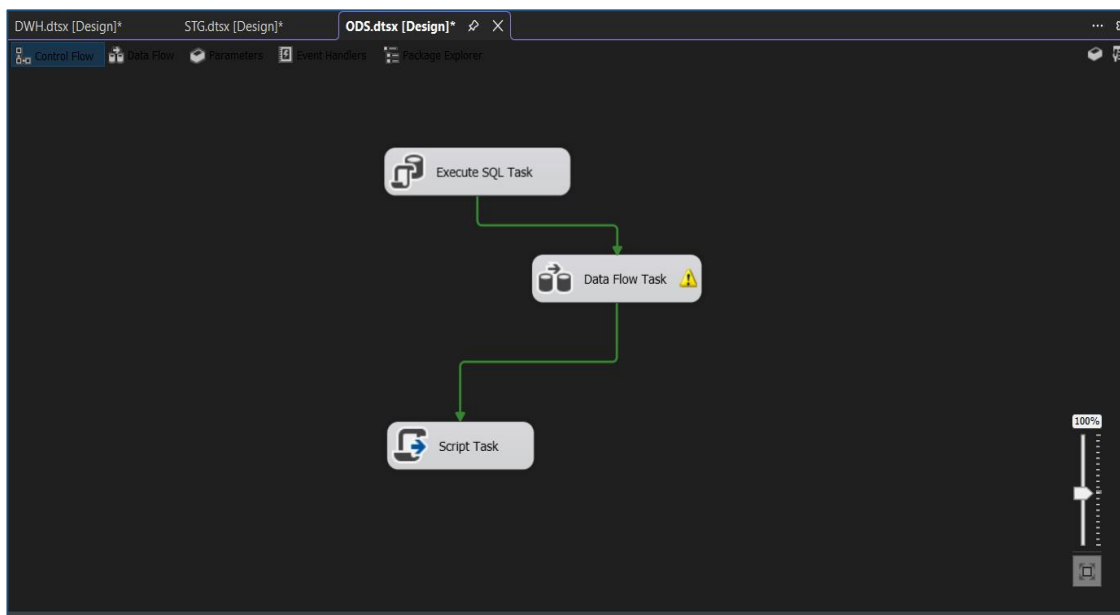
5. ETL Process (SSIS)

The ETL pipeline was built in SQL Server Integration Services across two SSIS packages — STG.dtsx and DWH.dtsx — implementing the classic Extract, Transform, Load pattern.

5.1 Extraction

Three SSIS source components extract data from the three source formats into the staging database:

- Flat File Source — reads MOCK_DATA.txt / MOCK_DATA.xlsx (CSV/Excel HR export), all columns mapped as NVARCHAR to avoid premature type conversion errors
- JSON Source — reads MOCK_DATA.json (Employee API feed), parsed and flattened into a tabular structure
- XML Source — reads dataset.xml (Legacy ERP export), validated against dataset.xsd before being accepted into staging

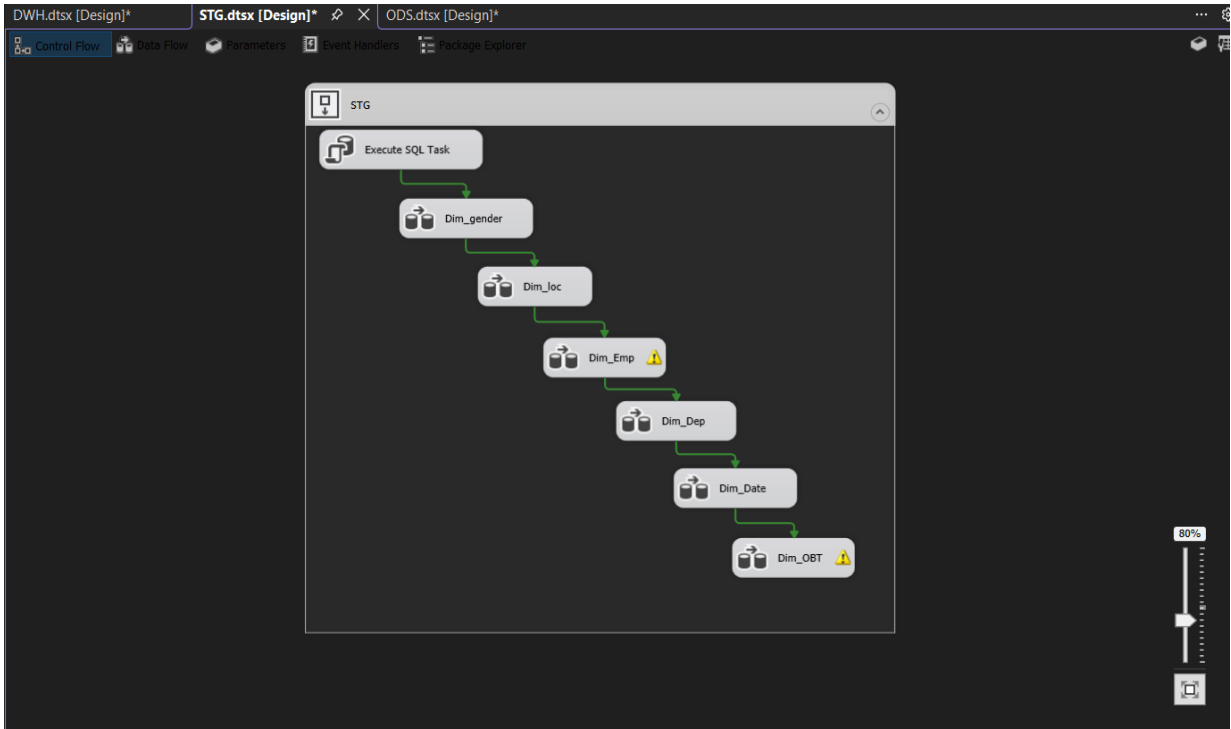


XSD Validation Step

The XML Source component is configured with dataset.xsd as its schema reference. Any XML record that does not conform to the expected structure or required fields is rejected at extraction and routed to the error output, preventing malformed records from ever reaching staging.

5.2 Transformation

All cleansing and standardization logic is applied in the data flow between the staging tables and the dimension/fact tables in DWH.dtsx. No transformations are applied during extraction — staging always preserves the raw values.



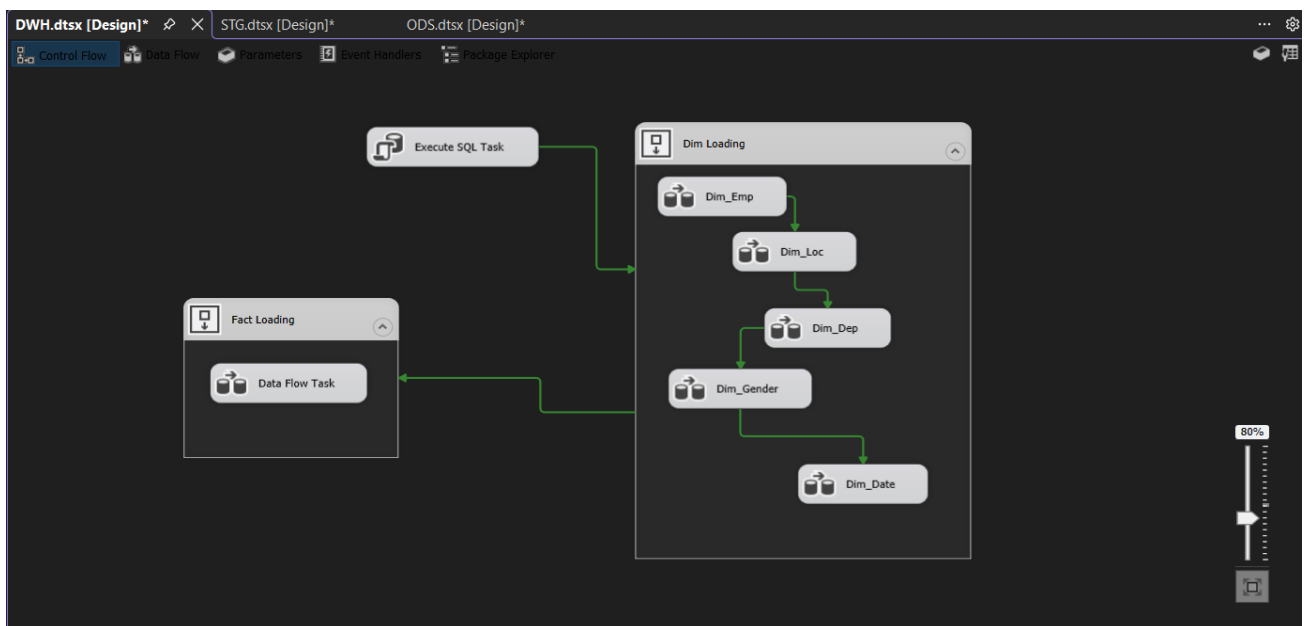
Transformation	From	To	SSIS Component
Column rename	countyr	country	Derived Column
Salary cleansing	\$3,206.23 (string)	3206.23 (decimal)	Derived Column + Data Conversion
Date normalization	DD.MM.YYYY / MM/DD/YYYY / DD.M.YYYY	YYYY-MM-DD	Derived Column (per-source expression)
Age calculation	birth_date	Age (int)	Derived Column
Gender standardization	M / F	Male / Female	Conditional Split + Derived Column

Transformation	From	To	SSIS Component
Name normalization	full_name (JSON only)	First_Name + Last_Name	Script Component
Surrogate key generation	Natural source keys	IDENTITY(1,1) keys	OLE DB Destination (Identity Insert)
Dimension lookups	Natural keys (country, dept, gender)	Fact table foreign keys	Lookup Transformation
Error routing	Invalid/malformed rows	Error log table	Lookup — No Match Output, Conditional Split

5.3 Loading

Loading follows a strict dependency order to preserve referential integrity — dimensions are always loaded before the fact table that references them:

1. Load DimEmployee from cleansed STG_Employees data (deduplicated by Email)
2. Load DimDep with distinct department names
3. Load DimLoc with distinct, standardized country names
4. Load DIMGender with the two standardized gender values
5. Load FactEmployeeSalary, using Lookup transformations against all four dimensions to resolve surrogate key foreign keys
6. Run post-load validation queries to confirm row counts and referential integrity



Load Strategy

Dimension tables use a full reload (truncate and reload) strategy due to the small, slowly-changing nature of department, location, and gender reference data. The fact table is loaded incrementally based on the staging load batch.

6. SQL Development

6.1 Stored Procedure — Dim_Date Population

A dedicated stored procedure generates and populates the DimDate dimension table, producing one row per calendar date across the required reporting range. This avoids hardcoding date logic into the ETL package and allows the date dimension to be regenerated independently.

```
CREATE PROCEDURE usp_Populate_DimDate
    @StartDate DATE,
    @EndDate DATE
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @CurrentDate DATE = @StartDate;

    WHILE @CurrentDate <= @EndDate
    BEGIN
        INSERT INTO DimDate (Date_Key, FullDate, [Year], [Quarter],
                             [Month], MonthName, [Day], DayName, WeekOfYear)
        SELECT
            CONVERT(INT, FORMAT(@CurrentDate, 'yyyyMMdd')),
            @CurrentDate,
            YEAR(@CurrentDate),
            DATEPART(QUARTER, @CurrentDate),
            MONTH(@CurrentDate),
            DATENAME(MONTH, @CurrentDate),
            DAY(@CurrentDate),
            DATENAME(WEEKDAY, @CurrentDate),
            DATEPART(WEEK, @CurrentDate);

        SET @CurrentDate = DATEADD(DAY, 1, @CurrentDate);
    END
END;
```

Execution:

```
EXEC usp_Populate_DimDate
    @StartDate = '1970-01-01',
    @EndDate   = '2030-12-31';
```

DimDate Output Columns

Column	Data Type	Description
Date_Key	INT PK	Surrogate key in YYYYMMDD format, matches Fact.Date_Key
FullDate	DATE	Calendar date
Year	INT	Calendar year
Quarter	INT	Calendar quarter (1–4)
Month	INT	Month number (1–12)
MonthName	VARCHAR(20)	Full month name
Day	INT	Day of month
DayName	VARCHAR(20)	Full weekday name
WeekOfYear	INT	ISO week number

Sample Validation Queries

```
-- Confirm fact table is loaded with clean, numeric salary
SELECT TOP 5 Salary FROM FactEmployeeSalary;

-- Confirm all foreign keys exist
SELECT name FROM sys.foreign_keys
WHERE parent_object_id = OBJECT_ID('FactEmployeeSalary');

-- Row count parity between STG and DWH
SELECT
    (SELECT COUNT(*) FROM STG_Employees_CSV) AS stg_count,
    (SELECT COUNT(*) FROM FactEmployeeSalary) AS dwh_count;
```

7. Challenges & Solutions

Building this pipeline surfaced a number of real-world data engineering challenges. Each is documented below along with the resolution applied.

Challenge	Impact	Solution Applied
Three different date formats across sources (DD.MM.YYYY, MM/DD/YYYY, DD.M.YYYY)	Silent data corruption if merged without normalization — e.g. 5/6 read as either May 6 or June 5	Per-source Derived Column expressions in SSIS normalize every date to YYYY-MM-DD before loading
countyr typo embedded in the source XSD schema	Propagated into all three datasets; downstream queries and joins would fail silently	Renamed at extraction via Derived Column; root-cause fix proposed in corrected XSD
Salary stored as a string with a \$ symbol	Impossible to aggregate, sort, or compare without casting	REPLACE() to strip the symbol, followed by CAST to DECIMAL(18,2) in the transformation layer
XSD declared every field as xs:string with minOccurs="0"	No type enforcement or required-field validation at the source — root cause of most quality issues	XML Source component validated against XSD; corrected schema proposed for future ingestion
DIMGender / dimension tables initially had no Primary Key defined	SSMS rejected foreign key relationship creation: "Primary key or UNIQUE constraint must be defined..."	Explicit PRIMARY KEY constraints added to all dimension tables before establishing foreign keys
JSON source used full_name instead of separate first/last name fields	Inconsistent schema compared to CSV and XML sources	Script Component splits full_name on the first space delimiter during transformation
Data entry error: stray @ symbol inside a last_name field (CSV row 288)	Would have corrupted the DimEmployee load if inserted directly	Row flagged by data profiling and routed to an error/quarantine table for manual review
Suspicious placeholder values in mock data (e.g. unusual test names)	Risk of polluting analytics with non-representative records	Flagged during profiling for business review prior to production cutover

8. Future Enhancements

While the current pipeline is functional end-to-end, the following enhancements are planned to bring the warehouse closer to a fully production-hardened state:

Priority	Enhancement	Description
High	Correct the source XSD schema	Fix countyr typo, type salary as xs:decimal, type age/date as xs:date, set minOccurs=1 on required fields
Medium	Finalize and integrate DimDate	Wire Date_Key in FactEmployeeSalary to the populated DimDate table for full time-based reporting
Medium	Implement SCD Type 2 on DimEmployee	Track historical changes to employee attributes (department transfers, name changes) over time
Medium	Move birth_date into DimEmployee	Age is currently stored as a static fact-table value; deriving it from a dimension-stored birth date is more correct
Low	Automated data quality checks in STG	Row-count thresholds, null checks, and type validation executed automatically after each staging load
Low	Power BI dashboard layer	Workforce analytics dashboard built directly on top of the Star Schema for self-service reporting
Low	Incremental loading	Move from full staging reloads to incremental/delta loads as data volume grows

9. Conclusion

This project demonstrates a complete, end-to-end data warehousing solution that addresses a problem nearly every organization faces: reconciling employee data scattered across incompatible source systems into one trustworthy, analytics-ready warehouse.

Starting from three structurally different sources — CSV, JSON, and XML — the pipeline applies a disciplined, layered architecture (ODS, STG, DWH) to preserve raw data integrity while systematically resolving more than ten distinct data quality issues, from a schema-level typo to inconsistent date formats across sources.

The resulting Star Schema — a single fact table connected to four dimension tables — gives analysts and BI tools a simple, query-friendly structure capable of answering core workforce questions: headcount by country, salary distribution by department, and demographic breakdowns by gender and age, all through straightforward joins rather than complex, error-prone data wrangling.

More broadly, this project reflects the core discipline of data engineering: profile before you transform, separate raw and cleaned data into distinct layers, fix problems as close to their root cause as possible, and design every table with the next analyst in mind. The patterns implemented here — multi-source ingestion, staged transformation, dimensional modeling — are directly transferable to HR, payroll, and workforce analytics challenges at organizations of any size.

Project Status

The ODS, STG, and DWH layers are implemented in SSIS with full referential integrity across the Star Schema. Future enhancements outlined in Section 8 will further harden the pipeline for sustained production use.